

UNIVERSITÉ IBN TOFAIL

FACULTÉ DES SCIENCES / ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES

PROJET FIN DE MODULE

API DE GESTION DE BIBLIOTHÈQUE

Conception, Conteneurisation Multi-stage et Déploiement
Continu de l'Application "LivreMoi"

Intitulé du Module : DevOps

Réalisé par l'étudiant : Alae Sassi

Date de rendu : Mai 2026

Année Universitaire : 2025 / 2026

Table des Matières

1. Introduction et Contexte du Projet	3
2. Architecture Globale et Choix Techniques	4
2.1 Architecture Logique N-Tier	4
2.2 Justifications des Choix Technologiques	4
3. Structure du Code et Modèle de Données	5
3.1 Organisation des Packages et Modules	5
3.2 Modèle Physique de Données (Oracle Database)	5
4. Fonctionnalités Applicatives Clés	6
5. Stratégie Git et Cycle de Vie DevOps	7
6. Conteneurisation Multi-stage de l'Application	8
6.1 Conception du Dockerfile Backend	8
6.2 Conception du Dockerfile Frontend	8
7. Orchestration Globale et Reverse Proxy	9
8. Conclusion et Perspectives	10
9. Annexes	11

1. Introduction et Contexte du Projet

Dans l'environnement académique et institutionnel moderne, la gestion efficace des ressources documentaires représente un pilier central pour la transmission du savoir. L'application "**LivreMoi**" a été conçue pour surmonter les limitations des anciens systèmes de gestion de bibliothèque, souvent caractérisés par des architectures monolithiques et des interfaces utilisateur rigides.

L'objectif principal de ce projet est de réaliser une refonte technologique complète en concevant une application web moderne reposant sur une **architecture découplée**. Le but n'est pas seulement de fournir une API fonctionnelle, mais de mettre en œuvre une démarche DevOps rigoureuse afin d'optimiser le cycle de développement, de garantir la portabilité des environnements et de fiabiliser les phases de déploiement en production.

Dans le cadre de ce module DevOps à l'Université Ibn Tofail, ce projet se focalise sur plusieurs axes méthodologiques et techniques :

- **Modularité Applicative** : Séparation stricte de la logique métier (Backend API REST) et de la couche de présentation (Frontend SPA).
- **Isolation et Portabilité** : Emploi de conteneurs Docker pour garantir que le système s'exécute à l'identique sur n'importe quelle machine hôte ("It works on my machine" resolution).
- **Optimisation des Builds** : Utilisation de builds multi-stage pour minimiser la surface d'attaque et l'empreinte mémoire des conteneurs applicatifs.
- **Persistence Enterprise Conservée** : Liaison directe avec l'instance Oracle Database physique du client afin de préserver l'ensemble des données d'utilisateurs et administrateurs existantes.

Objectifs Clés DevOps

L'application doit être instantanément fonctionnelle en local via une seule commande : `docker compose up`, tout en s'intégrant sans conflit à la base de données Oracle XE installée sur le système hôte via les adresses réseaux logiques de Docker.

2. Architecture Globale et Choix Techniques

2.1 Architecture Logique N-Tier

L'architecture de l'application LivreMoi adopte un paradigme découplé, séparant les responsabilités à travers un modèle à trois couches (3-Tier) :

```
[ Navigateur Web (Client React) ] --(Port 80/HTTP)--> [ Reverse Proxy Nginx ] | (Proxy Pass /api)
| v [ Base Oracle Physique Hôte ] <--(Port 1521)-- [ API REST Spring Boot (8080) ] (via
host.docker.internal)
```

Le frontend et le backend sont logés dans leurs conteneurs Docker respectifs, tandis que la base de données Oracle XE tourne nativement sur l'OS hôte du client. Le conteneur backend s'y connecte de manière transparente grâce au pont réseau virtuel de Docker (`host.docker.internal`).

2.2 Justifications des Choix Technologiques

- **Backend (Spring Boot 3.x / Java 17)** : Ce framework a été retenu pour sa maturité, sa robustesse dans le traitement transactionnel, sa sécurité native et sa couche d'abstraction ORM (Spring Data JPA / Hibernate) qui simplifie le dialogue avec des bases relationnelles d'envergure.
- **Frontend (React 19 / Vite)** : React permet de bâtir une SPA (Single Page Application) réactive, offrant une expérience utilisateur fluide. Vite a été choisi comme outil de build pour sa vitesse d'exécution lors du développement et sa capacité à générer un build de production statique hautement optimisé.
- **Base de Données Oracle XE Locale** : Le choix de maintenir l'accès à la base de données locale installée sur Windows garantit l'intégrité et la persistance des données historiques (comptes utilisateurs, administrateurs et historique des emprunts déjà configurés).

3. Structure du Code et Modèle de Données

3.1 Organisation des Packages et Modules

Le code source du backend est structuré selon les conventions Spring Boot pour séparer les concepts (Séparation des préoccupations) :

- `com.bibliotheque.entities` : Contient les classes de modèle persistantes mappées par Hibernate.
- `com.bibliotheque.repositories` : Fournit les interfaces étendant `JpaRepository` pour les opérations CRUD sur Oracle.
- `com.bibliotheque.service` : Implémente la logique métier (gestion des stocks lors des emprunts, vérification des retards).
- `com.bibliotheque.controller` : Définit les points d'entrée de l'API REST HTTP (gestion CORS intégrée).
- `com.bibliotheque.dto` : Gère la structure des données reçues et renvoyées (Data Transfer Objects).

3.2 Modèle Physique de Données (Oracle Database)

Le schéma relationnel est structuré autour de 5 tables clés. Pour assurer la compatibilité avec Oracle Database (notamment le respect de l'auto-incrémentation via séquences), nous utilisons des générateurs de séquence Hibernate configurés pour interagir directement avec les objets Oracle correspondants.

Table	Clé Primaire	Relations Clés	Séquence Oracle
USERS	id (RAW/NUMBER)	Aucune direct	USER_SEQ
AUTEUR	id (RAW/NUMBER)	1-N vers Livre	ETUDIANT_SEQ
CATEGORIE	id (RAW/NUMBER)	N-N avec Livre	ETUDIANT_SEQ
LIVRE	id (RAW/NUMBER)	N-1 Auteur	LIVRE_SEQ
EMPRUNTS	id (RAW/NUMBER)	N-1 User, N-1 Livre	EMPRUNT_SEQ

4. Fonctionnalités Applicatives Clés

L'application LivreMoi propose des fonctionnalités distinctes pour deux rôles principaux : les utilisateurs (lecteurs) et l'administrateur.

1. Authentification & Gestion des Droits

Un système de login et registre complet avec mot de passe masqué/visible. Un `AuthContext` React conserve le jeton utilisateur et son rôle, verrouillant l'accès aux pages selon les autorisations (ADMIN / USER). Les mots de passe sont encodés via `BCryptPasswordEncoder` dans le backend Spring Boot.

2. Espace Lecteur (Catalogue & Emprunts)

L'utilisateur dispose d'un tableau de bord personnel répertoriant ses emprunts en cours sous forme de liste paginée. Il peut naviguer dans le catalogue général, filtrer les livres par catégorie et effectuer un emprunt immédiat. Le système intègre **SweetAlert2** pour l'affichage de notifications dynamiques et de modales interactives de confirmation d'action.

3. Espace Administration & Gestion des Stocks

L'administrateur a le contrôle complet sur le catalogue physique :

- **Gestion du stock** : Toute action d'emprunt décrémente de manière atomique la quantité en stock dans la base Oracle. Réciproquement, un retour ou une annulation incrémente le stock.
- **Interface Admin dédiée** : Un volet d'administration permet de créer des livres, modifier leurs fiches, attribuer des catégories et valider le retour physique d'un livre emprunté.
- **Annulation de commande** : Les utilisateurs peuvent annuler un emprunt non validé, ce qui réajuste immédiatement le stock de livres.

5. Stratégie Git et Cycle de Vie DevOps

Une bonne gestion de versions est indispensable à la mise en œuvre de la démarche DevOps. Pour LivreMoi, Git a été utilisé de façon rigoureuse à travers un modèle de branches rationalisé afin de structurer les développements.

```
main (Production) ===== [v1.0.0] (Tag) ^ / | (Merge request / Pull) | v
develop (Intégration) ===== ^ ^ | (Merge) | (Branch &
Merge) | | feature/ (Fonctionnalité) +--[cors-fix]--++--[nginx-proxy]--
```

5.1 Modèle de Branches adopté

- **main** : Cette branche est sacrée. Elle ne contient que le code entièrement stable, testé et validé. C'est à partir de cette branche que sont tirées les images Docker destinées à être publiées sur le registre Docker Hub de production (`3alae008/api-gestion-bibliotheque-*`).
- **feature/ (branches thématiques)** : Pour chaque fonctionnalité ou correctif (par exemple, la correction des importations sensibles à la casse pour Linux, ou la mise en place de la configuration CORS), une branche dédiée est créée à partir de `main` (ou de la branche de développement) puis fusionnée après validation.

5.2 Politique de commits et conventions

Les messages de commits adoptent le format standardisé `Conventional Commits` (ex. : `feat: configurer la securite cors pour nginx`, `fix: corriger la casse de home.css pour le conteneur linux`). Cela facilite la génération automatique de changelogs et assure un historique clair pour l'équipe DevOps.

6. Conteneurisation Multi-stage de l'Application

Pour distribuer l'application de manière portable et performante, nous avons configuré des builds Docker basés sur le concept du **Multi-stage build**. Cette approche consiste à séparer l'environnement de compilation (lourd, contenant les outils de build) de l'environnement d'exécution (minimal, ne contenant que le strict nécessaire pour exécuter l'application).

6.1 Conception du Dockerfile Backend

Le Dockerfile du backend utilise deux étapes distinctes :

- **Étape 1 (Build)** : Basée sur l'image `maven:3.9.6-eclipse-temurin-17`. Cette étape récupère le code source, télécharge les dépendances définies dans `pom.xml` (mises en cache dans une couche intermédiaire pour accélérer les builds ultérieurs), puis compile l'application sous forme de fichier JAR exécutable.
- **Étape 2 (Runtime)** : Basée sur l'image ultra-légère `eclipse-temurin:17-jre-alpine`. Seul le fichier JAR produit à l'étape 1 y est copié. Le compilateur Maven, le JDK complet et le code source original sont ainsi exclus du conteneur final.

Avantages : La taille de l'image passe de près de 800 Mo à environ 180 Mo, réduisant la surface d'attaque de sécurité en éliminant les utilitaires inutiles en production.

6.2 Conception du Dockerfile Frontend

Pour l'application React/Vite, la même logique multi-stage est appliquée :

- **Étape 1 (Build)** : Basée sur `node:20-alpine`. Les dépendances npm sont installées via `npm ci`, et Vite compile l'application pour générer des fichiers statiques optimisés (HTML, JS, CSS) dans le dossier `/dist`.
- **Étape 2 (Serveur Web)** : Basée sur `nginx:alpine`. Les fichiers statiques y sont copiés. Nginx est configuré pour servir le frontend et agir comme reverse proxy.

7. Orchestration Globale et Reverse Proxy

7.1 Le Rôle de Nginx en Reverse Proxy

Pour éviter d'exposer directement le serveur d'application Spring Boot sur Internet et pour résoudre de manière définitive les blocages liés aux règles CORS du navigateur, un reverse proxy Nginx est configuré dans le conteneur du frontend.

Toutes les requêtes envoyées vers `http://localhost/` (port 80 par défaut) sont capturées par Nginx. Les requêtes pour l'interface statique sont servies directement. En revanche, les requêtes dirigées vers `/api` sont redirigées en arrière-plan vers le conteneur du backend à l'adresse `http://backend:8080`.

7.2 Orchestration avec Docker Compose

Le fichier `docker-compose.yml` coordonne l'infrastructure de production de manière simplifiée et sécurisée en instanciant deux services interconnectés :

1. **backend** : Compile et lance l'API Spring Boot. Il intègre des variables d'environnement pointant vers la base Oracle locale physique via l'adresse logique `host.docker.internal`. Le paramètre `extra_hosts` assure la résolution DNS de cette adresse sous Linux WSL2.
2. **frontend** : Distribue l'application React et expose le port 80 à l'utilisateur. Nginx s'y charge de l'acheminement des requêtes.

Avantage de la connexion hybride

En connectant le conteneur backend directement à la base Oracle hôte, nous n'avons pas besoin d'exécuter un conteneur Oracle XE volumineux en local, ce qui économise de la mémoire RAM et CPU sur la machine de développement tout en conservant les données d'administration d'origine.

8. Conclusion et Perspectives

Le projet "LivreMoi" a permis de mettre en place une solution web moderne, robuste et industrialisée de gestion de bibliothèque, combinant la puissance de Spring Boot, la réactivité de React et la fiabilité d'Oracle Database.

Sur le plan DevOps, la conteneurisation intégrale de l'application et de son système de persistance avec Docker et Docker Compose répond aux standards modernes de portabilité et de reproductibilité. Les problématiques récurrentes en développement web, telles que la gestion des CORS et la configuration fine des environnements de base de données, ont été adressées de manière transparente à l'aide de configurations reverse proxy Nginx et de variables d'environnement.

De plus, les images optimisées produites par les builds multi-stage ont été publiées avec succès sur le registre public Docker Hub (sous le compte `3a1ae008`), rendant le déploiement en production accessible en une seule commande depuis n'importe quel serveur distant.

Perspectives d'Évolutions DevOps

Pour poursuivre cette dynamique DevOps, plusieurs axes d'améliorations peuvent être envisagés :

- **Pipeline CI/CD complète** : Intégration de GitHub Actions ou GitLab CI pour exécuter automatiquement les tests unitaires et d'intégration à chaque commit, puis builder et pousser automatiquement les nouvelles images étiquetées sur Docker Hub.
- **Monitoring applicatif** : Intégration de Spring Boot Actuator avec Prometheus et Grafana pour collecter et visualiser des métriques système (utilisation CPU, mémoire, temps de réponse des requêtes) et anticiper les pannes.
- **Orchestration de production** : Migration de la configuration Docker Compose vers un cluster Kubernetes (K8s) pour gérer la haute disponibilité de l'application et sa mise à l'échelle automatique (autoscaling).

9. Annexes - Extraits de Code (1/2)

Fichier docker-compose.yml de l'application (Connexion à la base hôte)

```
version: '3.8'

services:
  # Backend Service: Spring Boot Application connecté à l'instance Oracle locale de l'hôte
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: library-backend
    ports:
      - "8080:8080"
    environment:
      # Connexion à l'instance Oracle locale (hôte Windows) via host.docker.internal
      - SPRING_DATASOURCE_URL=jdbc:oracle:thin:@host.docker.internal:1521:xe
      - SPRING_DATASOURCE_USERNAME=library
      - SPRING_DATASOURCE_PASSWORD=sys
      - SPRING_JPA_HIBERNATE_DDL_AUTO=update
    extra_hosts:
      - "host.docker.internal:host-gateway"
    restart: always

  # Frontend Service: React App + Nginx Reverse Proxy
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: library-frontend
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always
```

9. Annexes - Extraits de Code (2/2)

Configuration Reverse Proxy Nginx (frontend/nginx.conf)

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }

    # Proxy API requests to the backend service
    location /api {
        proxy_pass http://backend:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root /usr/share/nginx/html;
    }
}
```

Références bibliographiques

- Documentation officielle de Docker et Docker Compose : <https://docs.docker.com>
- Guide de démarrage rapide de Spring Boot et JPA : <https://spring.io/projects/spring-boot>
- Documentation de React 19 et Vite.js : <https://vite.dev>
- Résolution DNS Docker pour hôte WSL2 : <https://docs.docker.com/desktop/networking/>